eVTOL Simulation Problem

Design Choices

I started with a simple init, step, destroy set of functions in a simulation class and added data structures for the various parameters in `types.h`. I took a state-machine approach to the various vehicle states, airborne, ground, and charging. For the chargers, I added a FIFO queue to allow vehicles to maintain sequencing. This made the ground state a bit more involved than the other states, so I split the code into further functions. I reused queue and CSV parsing code I had written previously, although I had not yet placed those utilities into online source control, which I wanted to do anyway.

In terms of libraries, I tend to favor C stdio printf functions over the C++ `iostream` library. Setting various formatting options with `iomanip` has never struck me as particularly elegant, although C++20 `std::format()` provides functionality somewhat similar to Python f-strings, which seems less cumbersome.

I did the same with file operations, sticking with C library style versus C++ streams, although opening files with C++ streams is not particularly cumbersome.

The only allocations occur in `init()`, and the only deallocations occur within `destroy()`, although the CSV parsing utilities used by `main()` internally perform heap allocations. I was tempted to support a dynamic amount of rows in the CSV files, but that would require an additional dynamic allocation, so I opted to keep the CSV row limits statically defined.

I stuck with C++ `new`/`delete` for memory allocation and relied on the default exception behavior of terminating the program if an allocation failure occurs. If I were using malloc and free I would check for NULL, then use perror to print error information with the errno value, and then return failure codes back to main, which usually would then exit anyway.

I kept things single threaded, although I would probably use OpenMP-style parallel-for operations to allow the main loop to simulate each vehicle in parallel. The primary issue would be the charging stations causing significant contention. Obviously we could use semaphores or mutexes to synchronize the threads. However, without careful planning, the synchronization overhead could eliminate most performance gains and leave threads waiting on shared resources. I have seen false sharing occur even in workloads that were otherwise perfectly parallel, so having results written to common memory can cause slowdowns that are not obvious due to shared cache lines.

For unit testing I used Google Test, which works easily enough in a Linux environment. Since I initially wrote the code in Visual Studio, once most of the functionality was working I moved the project to an AWS Linux instance, added it to Git, created Makefiles, and integrated a basic unit testing framework. I decided to use Doxygen-style function and file headers also, so at this point I added those to the codebase. I am not particularly fond of writing extensive function and file headers, so I used AI assistance to help generate initial documentation headers, which were then manually reviewed and refined before integration into the codebase.

In terms of function headers I have not been a huge fan of maintaining duplicated documentation between `.h` and `.cpp` files. Additionally, I feel that function headers can obscure the implementation when you just want a high-level overview of a class. So I opted to just add function headers to the .cpp files, and only use file headers in the .h files.

Since the simulation is driven by CSV input files, I can use different input files to test using the same test runner, whereas traditionally I would test specific functions or classes in isolation. My main concern regarding testing is really that the statistics generated at the end are accurate. So I can test this using different input files and validate that the outputs make sense. For example, one could configure a cruise speed of 100 mph, a battery capacity of 100 kWh, a one-hour charge time, and an energy usage of one kWh per mile. This keeps the numbers relatively round while also ensuring all vehicles share identical parameters so the test focuses on a single use case.

This problem is a bit open ended, so one could easily spend a lot of time working on things. That said, this week I was a bit busy, wrote most of the code on Tuesday afternoon, got it on GitHub Wednesday afternoon and left it to sit until the weekend to allow for additional unit tests and writing this documentation due to other obligations. I think it would be nice to have latitudes and longitudes, a visualization of some sort (Google Earth, Cesium, or similar) and perhaps even some gravity, thrust, and gyroscopic stabilization. But I felt a relatively bare-bones implementation was sufficient for the scope of this exercise.